

Technologies DevOps

Github actions

Bases du versionning

L'importance du versionning ?

- Créer une sécurité de donnée sur le code
- Pouvoir revenir en arrière facilement
- Gérer plusieurs branches d'un même code
- Partager et travailler sur un même projet à plusieurs

Quid du cloud versionning ?

In case of fire



 1. **git commit**

 2. **git push**

 3. **leave building**

Avant git ?

Dans les temps ancestraux, comment faisaient les développeurs ?

Linus Torvalds n'a lancé la première version de Git qu'en 2005, et Github est né en 2008

Avant son apogée à travers tous les OS, Subversion (Apache) était l'outil de versionning le plus utilisé. Il existe encore aujourd'hui mais est beaucoup moins populaire

The battle

Git	vs	SVN
Used by ~90% of professional developers		Used by ~10% of professional developers
Distributed Version Control		Centralized Version Control
Can work locally, offline		Must be connected to commit
Each user has a copy of the full repository		Each user only has a copy of the trunk
Easy to fork, branch, and merge		Branching and merging is time-consuming

Commandes GIT de base

- `git config` //définir les infos de l'utilisateur (mail, nom)
- `git init` //initialiser un projet git
- `git clone` //copier en local un projet git
- `git add` //ajouter des fichiers dans le prochain commit
- `git commit` //créer un point de sauvegarde
- `git status` //s'assurer des fichiers en attente de commit
- `git log` //lister les commits effectués
- `git reflog` //garde l'historique local privé complet
- `git pull` //charger en local les modifications en ligne
- `git push` //envoyer ses commits en ligne

Les branches

Les branches permettent de gérer des états et modifications différents d'un même projet

- `git branch` //lister, créer ou supprimer des branches
- `git checkout` //créer des branches ou basculer entre elles
- `git merge` //fusionner 2 branches de code
- `git rebase` //déplacer ses commits d'une autre branche
- `git diff` //lister les conflits entre branches
- `git fetch` //récupérer les modifications de tout le repo
- `git cherry-pick` //prends un commit spécifique et le copie d'une autre branche

git fetch & git pull

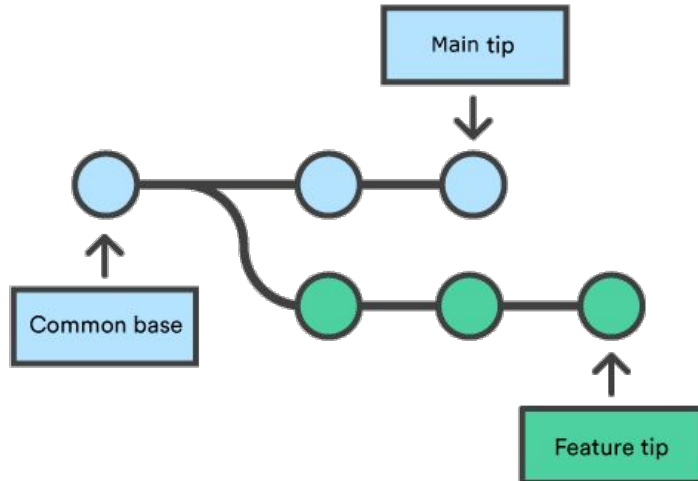
Assurez-vous que la branche cible et la branche mergée sont mises à jour avec les derniers changements distants. Exécutez la commande `git fetch` pour faire un pull des derniers commits distants. Une fois que le fetch est terminé, assurez-vous que la branche principale (main) est à jour en exécutant la commande `git pull`.

git merge

git merge combinera plusieurs séquences de commits en un historique unifié. Dans les cas d'usage les plus fréquents, git merge est utilisée pour combiner deux branches.

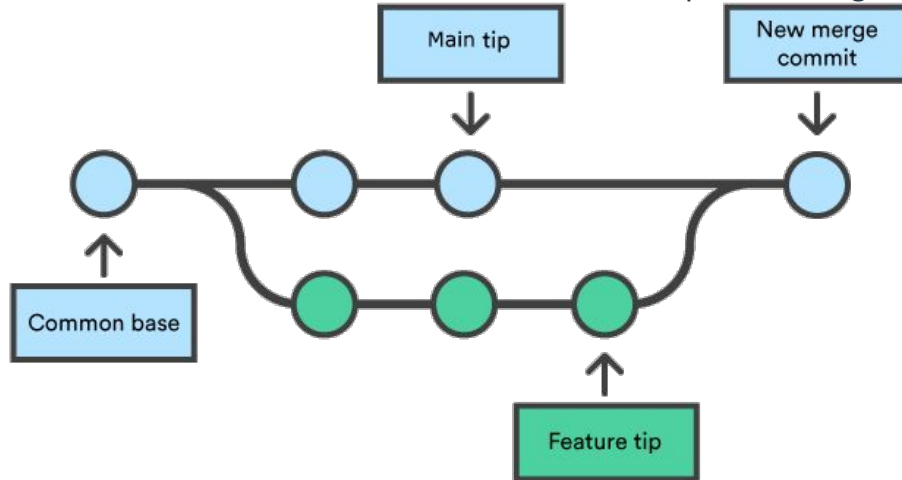
git merge

Supposons que nous ayons une nouvelle fonctionnalité de branche basée sur la branche (main). Nous souhaitons maintenant faire un merge de cette branche de fonctionnalité dans la branche (main).



git merge

Appeler cette commande permettra de merger la fonctionnalité de branche spécifiée dans la branche actuelle, disons main. Git déterminera automatiquement l'algorithme de merge



git merge

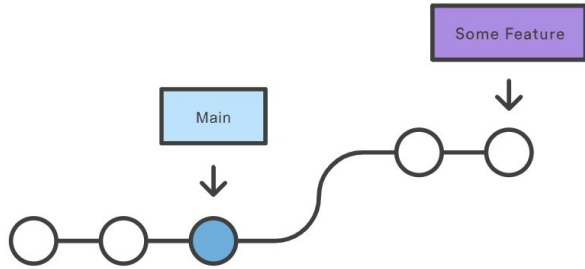
Les commits de merge sont uniques par rapport aux autres commits, car ils ont deux commits parent. En créant un commit de merge, Git essaiera de merger automatiquement les historiques distincts à votre place. Si Git rencontre des données différentes dans les deux historiques, il ne pourra pas les combiner automatiquement. Ce scénario constitue un conflit de contrôle de version, et l'intervention d'un utilisateur sera nécessaire pour continuer.

git merge fast-forward

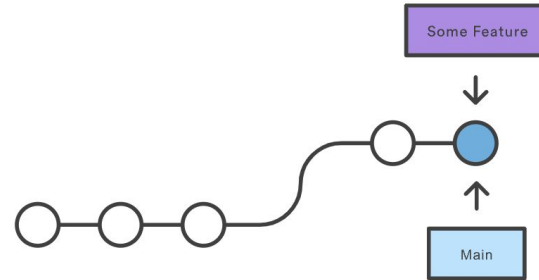
Un fast-forward merge peut avoir lieu lorsque le chemin entre la pointe de la branche actuelle et la branche cible est linéaire. Plutôt que de réellement merger les branches, Git doit simplement déplacer la pointe de la branche actuelle vers celle de la branche cible pour intégrer les historiques. Cette opération est possible, puisque tous les commits accessibles à partir de la branche cible le deviennent également via la branche actuelle.

git merge fast-forward

Before Merging



After a Fast-Forward Merge

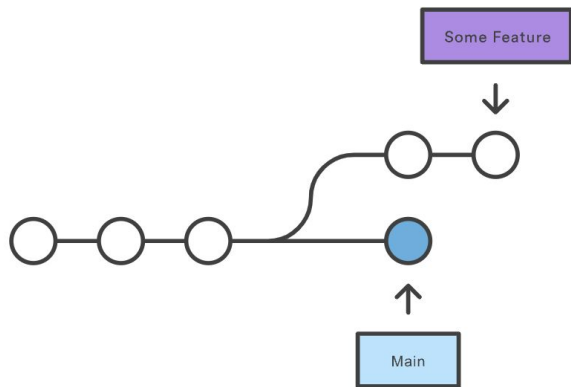


git merge --no-ffw

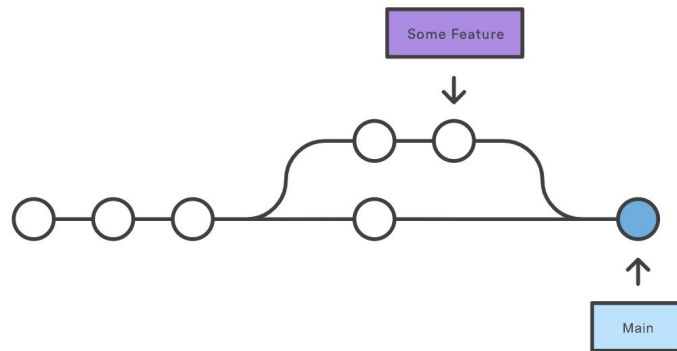
Toutefois, un fast-forward merge n'est pas possible si les branches ont divergé. Lorsque le chemin vers la branche cible n'est pas linéaire, Git n'a d'autre choix que de combiner les branches avec un merge à trois branches. Les merges à trois branches utilisent un commit dédié pour lier deux historiques. La nomenclature vient du fait que Git utilise trois commits pour générer le commit de merge : les deux pointes de branche et leur ancêtre commun.

git merge --no-ffw

Before Merging



After a 3-way Merge



Les conflits

Si les deux branches que vous essayez de merger modifient toutes les deux la même partie du même fichier, Git ne peut pas déterminer la version à utiliser. Lorsqu'une telle situation se produit, Git s'arrête avant le commit de merge, afin que vous puissiez résoudre manuellement les conflits.

L'avantage du merge dans Git est que le workflow habituel d'édition, d'indexation et de commit est utilisé pour résoudre les conflits de merge. Lorsque vous faites face à un conflit de merge, lancez la commande `git status` pour afficher les fichiers en conflit.

Résolution de conflit

here is some content not affected by the conflict

<<<<<< main

this is conflicted text from main

=====

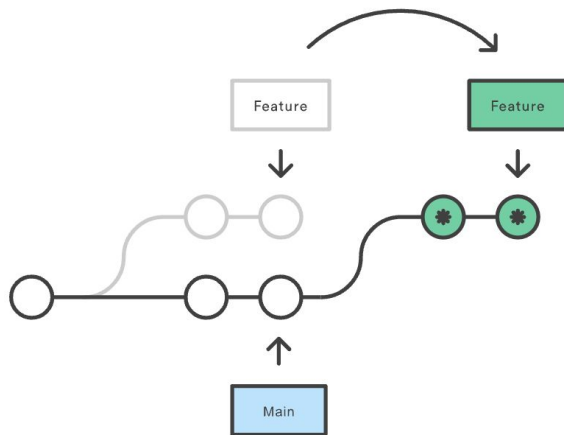
this is conflicted text from feature branch

>>>>>> feature branch;

En principe, le contenu situé avant le marqueur ===== représente la branche cible et la partie après ce marqueur représente la branche mergée. Une fois les fichiers modifiés, vous pouvez exécuter `git add` sur les fichiers en conflit, puis `commit`.

git rebase

Le rebase consiste à déplacer vers un nouveau commit de base ou à combiner une séquence de commits. Le rebasage est plus utile et facilement visible dans le contexte d'un workflow de branche de fonctionnalité.



git rebase --interactive

Exécuter git rebase avec l'option -i démarre une session de rebasage interactive. Au lieu de déplacer aveuglément tous les commits vers la nouvelle base, le rebasage interactif vous permet de modifier des commits un à un au cours du processus. Vous pouvez ainsi nettoyer l'historique en supprimant, en séparant et en modifiant une série existante de commits.

git rebase --interactive

Rebase 2cf755d..ee2adc2 onto 2cf755d (9 commands)

Commands:

p, pick = use commit

r, reword = use commit, but edit the commit message

e, edit = use commit, but stop for amending

s, squash = use commit, but meld into previous commit

f, fixup = like "squash", but discard this commit's log message

x, exec = run command (the rest of the line) using shell

d, drop = remove commit

git cherry-pick

git cherry-pick est une commande puissante qui permet de choisir des commits Git arbitraires par référence et de les ajouter au HEAD actuel. Elle correspond au fait de sélectionner un commit d'une branche et de l'appliquer à une autre. La commande git cherry-pick peut être utile pour annuler des changements. Par exemple, supposons qu'un commit soit accidentellement intégré à la mauvaise branche. Vous pouvez passer à la bonne branche, sélectionner le commit et l'insérer à l'endroit où il devrait se trouver.

Quand utiliser cherry pick ?

git cherry-pick est un outil utile, mais pas toujours une bonne pratique. La sélection de commits peut entraîner des doublons, et les merges traditionnels lui sont souvent préférés. Ceci dit, la commande git cherry-pick est pratique dans certains cas...

Collaboration entre deux équipes

Il arrive souvent que les membres individuels d'une équipe travaillent dans le même code ou autour de celui-ci. Il se peut qu'une nouvelle fonctionnalité du produit comporte un composant back-end et un composant front-end. Il est possible que du code soit partagé entre deux secteurs de produits. Le développeur du back-end peut créer une structure de données que le front-end devra également utiliser. Le développeur du front-end pourrait utiliser la commande git cherry-pick pour sélectionner le commit dans lequel cette hypothétique structure de données serait créée. De la sorte, le développeur du front-end pourrait continuer à avancer dans sa partie du projet.

Hotfix de bug

Lorsqu'un bug est découvert, il est important de fournir une correction aux utilisateurs finaux aussi rapidement que possible. Par exemple, un développeur a commencé à travailler sur une nouvelle fonctionnalité. Au cours du développement de cette nouvelle fonctionnalité, il identifie un bug préexistant. Il crée un commit explicite pour le corriger. Le commit de ce nouveau patch peut être directement sélectionné dans la branche principale (Main) pour corriger le bug avant qu'il ne touche plus d'utilisateurs.

Annulation de changements et restauration de commits perdus

Parfois, une branche de fonctionnalité (Feature) peut devenir obsolète et ne pas être mergée dans la branche principale (Main). Parfois, une pull request peut être fermée sans être mergée. Git ne perd jamais ces commits, et il reste possible de les rechercher et de les ramener à la vie grâce à des commandes comme `git log` et `git reflog`.

Quelques commandes supplémentaires

- `git tag` //créer une étiquette (pour les versions)
- `git reset` //supprime les modifications
- `git stash` //met de côté les fichiers sans les commits
- `git grep` //faire une recherche

Outil git

- git bash
- git lens + git graph (extension vscode)
- gitkraken (payant)
- github desktop
- sourcetree (svn compatible)
- tortoisegit (windows only)

Hébergeur de codes

- github
- gitlab
- bitbucket
- aws codecommit
- azure repos

Github Pages

Github mets à disposition un petit outil de déploiement gratuit

- Les dépôts sources GitHub Pages ont une limite recommandée de 1 Go.
- Les sites publiés GitHub Pages ne peuvent ne pas avoir une taille supérieure à 1 Go.
- Les déploiements GitHub Pages expirent s'ils prennent plus de 10 minutes.
- Les sites GitHub Pages ont une limite de bande passante souple de 100 Go par mois.
- Les sites GitHub Pages ont une limite souple de 10 builds par heure. Cette limite ne s'applique pas si vous générez et publiez votre site avec un workflow GitHub Actions personnalisé

GitHub Actions

GitHub Actions aide les développeurs à automatiser les tâches du cycle de vie du développement logiciel. Ces actions sont basées sur des événements.

Free

The basics for individuals
and organizations

\$ 0

per month
forever

Create a free organization

- > **Unlimited public/private repositories**
- > **Automatic security and version updates**
- > **2,000 CI/CD minutes/month**
Free for public repositories
- > **500MB of Packages storage**
Free for public repositories
- > **Issues & Projects**
- > **Community support**

FEATURED ADD-ONS

- > **GitHub Copilot Access**
- > **GitHub Codespaces Access**

Git Workflow

Un workflow Git est un processus automatisé configurable qui peut exécuter une ou plusieurs tâches.

Pourquoi avoir plusieurs workflow ?

Workflow de développement vs workflow de production

Vos clés et dépendances API peuvent être différentes

Créer un workflow

Dans cette section, nous allons configurer un workflow npm pour notre projet React.

1. Aller sur l'onglet Actions du repo git
2. Clic sur New Workflow
3. Clic sur "set up a workflow yourself"

set a workflow yourself

The screenshot displays the GitHub web interface for a repository named 'ci-cd-ynov' by user 'Loise'. The top navigation bar includes links for Code, Issues, Pull requests, Actions, Projects, Wiki, Security, and Insights. The main content area shows the workflow editor for the file 'main.yml' in the 'master' branch. The editor has tabs for 'Edit' and 'Preview', and a toolbar with options for 'Spaces', '2' (line numbers), 'No wrap', and a 'Copy' icon. The text area contains a single line: '1 | Enter file contents here'. On the right side, there is a 'Marketplace' section with a search bar and a 'Featured Actions' list. The first action listed is 'Setup Node.js environment' by 'actions', which has 3.4k stars. The description for this action reads: 'Setup a Node.js environment by adding problem matchers and optionally downloading and adding it to the PATH'.

Loise / ci-cd-ynov

<> Code Issues Pull requests Actions Projects Wiki Security Insights

ci-cd-ynov / .github / workflows / main.yml in master

Cancel changes Commit changes...

Edit Preview Spaces 2 No wrap

1 | Enter file contents here

Marketplace Documentation

Search Marketplace for Actions

Featured Actions

 **Setup Node.js environment** ☆ 3.4k

By actions

Setup a Node.js environment by adding problem matchers and optionally downloading and adding it to the PATH

Mettre à jour le yaml

Renommer le fichier `build_test.yml`

Et éditer le fichier avec en première ligne :

```
name: Build and Test Application
```

Choisir le trigger

Controls when the action will run.

on:

Triggers the workflow on push or pull request events but only for the main branch

push:

branches: [master]

pull_request:

branches: [master]

Les jobs

Une exécution de flux de travail est composée d'une ou plusieurs tâches appelés jobs.

Chaque tâche s'exécute dans un environnement d'exécution spécifié par runs-on.

Vous pouvez exécuter un nombre illimité de tâches tant que vous respectez les limites d'utilisation du flux de travail.

Initialiser un job pour les tests

A workflow run is made up of one or more jobs that can run sequentially or in parallel

jobs:

build_test:

The type of runner that the job will run on

runs-on: ubuntu-latest

Les steps

Un job contient une séquence de tâches appelées steps (étapes).

Les steps peuvent exécuter des commandes, exécuter des tâches de configuration ou exécuter une action dans votre repo, un repo public ou une action publiée dans un registre Docker.

Chaque step s'exécute dans son propre processus au sein de l'environnement d'exécution et a accès à l'espace de travail et au système de fichiers.

Étant donné que les étapes s'exécutent dans leur propre processus, les modifications apportées aux variables d'environnement ne sont pas conservées entre les étapes.

Créer un step qui va build les images

steps:

- name: Checkout code

 - uses: actions/checkout@v4

- name: Set up Docker Buildx

 - uses: docker/setup-buildx-action@v3

- name: Build image for scanning

 - run: docker build -t my-app:1.0.0 -f ./Dockerfile .

Créer un step qui vérifier ces images

- name: Docker Scout Scan

uses: docker/scout-action@v1

with:

command: cves

image: my-app:1.0.0

file: ./Dockerfile

only-severities: critical,high

exit-code: true

Executer le workflow

1. commit le fichier
2. pusher le fichier
3. aller dans actions
4. suivre les jobs

Commit changes

Commit message

Create build_test_react.yml

Extended description

Add an optional extended description..

Commit Email

- ☒ Commit directly to the master branch
- ☐ Create a **new branch** for this commit and start a pull request

[Learn more about pull requests](#)

Cancel

Commit changes

Ajouter la dépendance aux tests

Ajouter une dépendance au job `build_test` pour ne s'exécuter qu'en cas de succès

Add a dependency to the build job

```
needs: build_test
```

docker & github

On va tester le déploiement de notre application dans un docker compose au sein de notre pipeline github

- name: Build the stack

run: docker compose up -d

L'objectif ? Pouvoir exécuter des test end2end sur notre architecture

Et à terme, déployer cette architecture vers un service d'hébergement docker payant comme digitalOcean, aws, hostinger, kamatera, ou vers un serveur perso

A vous de jouer

A la suite du lancement du docker compose, vous allez créer une action qui va vérifier l'état des composants et du statut healthy de chaque container

Exemple de déploiement avec aws ec2

```
name: Deploy
on:
  push:
    branches: [ main ]

jobs:
  EC2-Deploy:
    runs-on: ubuntu-latest
    environment:
      name: ${{ github.ref_name }}
      url: ${{ steps.deploy.outputs.vm_url }}
    steps:
      - id: deploy
        name: Deploy
        uses: bitovi/github-actions-deploy-docker-to-ec2@v0.5.0
        with:
          aws_access_key_id: ${{ secrets.AWS_ACCESS_KEY_ID }}
          aws_secret_access_key: ${{ secrets.AWS_SECRET_ACCESS_KEY }}
          aws_default_region: us-east-1
          app_port: 3000
```

source : <https://www.bitovi.com/blog/deploy-any-docker-project-to-aws-with-github-actions>